SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks



Lecture 08

Git, Collaborative Development, CI/CD, Security & Code Quality

Tools: Git · GitHub · GitHub Actions

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

Earlier	L07	TODAY	Next	Final
React · C#/.NET · DB · Auth	Agentic & AI-assisted dev	Git, CI/CD, security & quality	Deployment & DevOps	Project showcase

- **Module content area: professional engineering practices, tooling and delivery**
- You can now BUILD a full-stack app — today is about building it as a TEAM, safely and repeatably
- Git history, pull requests and CI/CD are graded components of the Main Assignment
- Foundation for the next lecture's deployment and DevOps topics

Lecture Learning Outcomes

LO3

Use best practices for integration & quality

Apply Git workflows, pull-request code review, CI/CD automation, clean-code and secure-coding practices to deliver reliable software as a team.

LO4

Select & justify approaches

Choose and justify a branching strategy, a CI/CD pipeline design, and security & dependency-management practices appropriate to a project.

Why Git, CI/CD, Security & Quality Matter

Software is a team sport

Real products are built by teams over years. Without version control and a shared workflow, teams overwrite work and block each other daily.

Mistakes must be reversible

Git is a time machine: every change is tracked, attributable and revertible. 'It worked yesterday' becomes a one-command rollback.

Ship fast, ship safely

CI/CD automates build, test, quality and security checks on every change — catching defects in minutes, not in production.

Security is everyone's job

Leaked keys, injection and vulnerable dependencies cause real breaches. Secure habits + automated scanning prevent expensive incidents.

Lecture Roadmap – 3 Hours

1	**Version control with Git & GitHub** Repos · commits · branches · the everyday workflow	~30 min
2	**Branching strategies & collaboration** GitFlow · GitHub Flow · trunk-based · working in teams	~30 min
3	**Pull requests, code review & issues** PR workflow · review checklist · merge conflicts · issue tracking	~30 min
4	**CI/CD & automated build/testing** CI vs CD vs CD · pipeline stages · GitHub Actions · automated tests	~35 min
5	**Code quality & security** Clean code · linting/SonarQube · security risks · secure coding · dependencies	~45 min

PART 1

Version Control with Git & GitHub

- What version control is and why it matters
- Git vs GitHub; local vs remote repositories
- Core concepts: repo, commit, branch, merge, clone, pull, push
- The everyday workflow & good commit practices

What Is Version Control — and Why?

- **Version control = a system that records changes to files over time**
  - Every change is saved as a snapshot you can return to
  - Answers: what changed, when, by whom, and why
- **Why it is essential in software engineering:**
  - Full history & instant rollback — undo any change safely
  - Parallel work — many developers on one codebase without chaos
  - Accountability & traceability — link every line to a reason
  - A safety net that enables fearless experimentation

✗ Without version control

```
report_final.docx
report_final_v2.docx
report_FINAL_use_this.docx
report_final_v2_amal_edit.docx
```

✓ With Git

One project, full timeline

Each commit = a labelled save point

See diffs between any versions

Many people, one source of truth

Git vs GitHub — Not the Same Thing

Git

- **The version-control TOOL itself**
- Runs locally on your machine
- Tracks changes, branches, merges, history
- Created by Linus Torvalds (2005)
- Works with no internet, no account
- Command line or built into IDEs

GitHub

- **A cloud PLATFORM that hosts Git repos**
- Adds collaboration on top of Git
- Pull requests, code review, issues
- CI/CD via GitHub Actions, security tools
- Owned by Microsoft; needs an account
- Alternatives: GitLab, Bitbucket

Analogy: Git is the engine; GitHub is the garage where teams park, share and service their cars. You can use Git alone — but GitHub is where teams collaborate. (GitLab and Bitbucket are competing garages.)

Git — A Distributed Version Control System

Git is distributed: every developer has a complete local copy of the project and its history.

Full local history

Work offline, inspect history and recover the project from any complete clone.

Fast local operations

Commit, branch, diff and log are quick because most work happens locally.

Branch-friendly

Create feature branches, tags and releases without affecting main.

HTTPS / SSH

Connect securely to remotes such as GitHub, GitLab, Bitbucket or Azure Repos.

Local commits

Save work locally first; push only when ready to share with the team.

Stashing

Temporarily store unfinished work, switch tasks, then re-apply it later.

Student example: commit your lab work locally even without internet, then push to GitHub when you reconnect.

Git & GitHub Alternatives

GitHub is common, but the concepts transfer across tools and platforms.

Version-control alternatives

Fossil — DVCS with built-in issue tracking and wiki

Mercurial — distributed VCS focused on performance and simplicity

Subversion (SVN) — centralized VCS still used in legacy projects

Repository hosting platforms

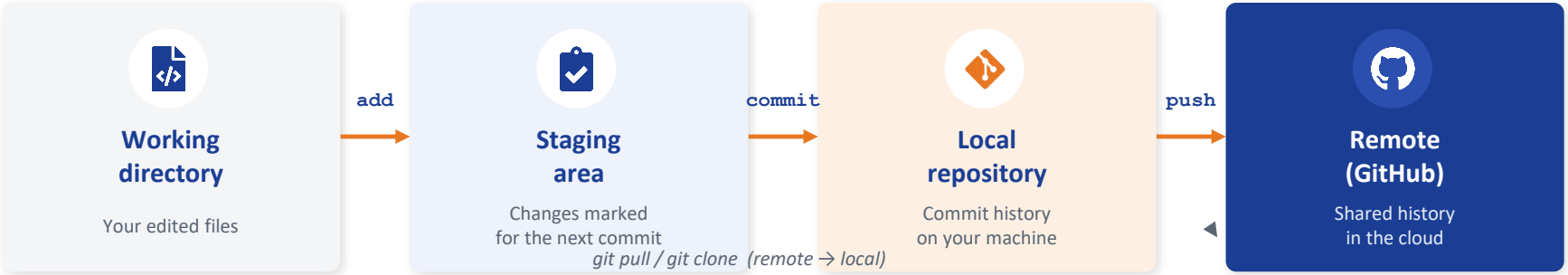
GitLab — Git hosting with integrated CI/CD and DevOps tooling

Bitbucket — integrates well with Atlassian tools

Azure Repos / AWS CodeCommit — cloud-provider repository services

Key point: learn the workflow — repositories, branches, commits, PRs and CI/CD — not only one website.

Local vs Remote Repositories



- **Local repository — the full project + history on YOUR computer; works offline**
- **Remote repository — the shared copy on GitHub the team syncs with**
- git clone copies a remote to your machine; git push uploads commits; git pull downloads teammates' commits
- **Git is distributed: everyone has the complete history, not just a central server**

Staging in Git — The Middle Step

Staging is the middle step between editing files and saving a commit.

1. Working directory

You edit files here.

Changes are not yet part of Git history.

git add

2. Staging area

git add selects changes for the next commit.

Only selected files are included.

git commit

3. Repository

git commit records the staged snapshot.

The commit becomes part of local history.

Why stage?

Review first • group related changes • avoid committing unwanted files • keep each commit focused

Core Git Concepts at a Glance

Term	What it means	Command
Repository	A project tracked by Git, including its full history	git init / git clone
Commit	A saved snapshot of changes with a message & author	git commit -m "..."
Branch	An independent line of work that can diverge from main	git branch / git switch
Merge	Combining the changes from one branch into another	git merge <branch>
Clone	Make a local copy of a remote repository	git clone <url>
Pull	Fetch + merge the latest commits from the remote	git pull
Push	Upload your local commits to the remote	git push

Basic Git Commands — Local Work

These commands cover the basic local Git workflow.

Initialize

git init
Start tracking an existing local project.

Clone

git clone <url>
Copy an existing repository and full history.

Stage

git add <file> or git add .
Prepare selected changes for commit.

Commit

git commit -m "Fixed navbar"
Save a local snapshot with a message.

Status / Log

git status
git log --oneline
Inspect current state and history.

Remember: a commit is local until you push it to the remote repository.

Basic Git Commands — Branches & Remotes

These commands help students work with branches and shared remote repositories.

Branching & merging

git branch <name> — create a branch

git switch <name> — move to a branch

git switch -c <name> — create and switch

git merge <name> — merge into current branch

git branch -d <name> — delete merged branch

Working with remotes

git remote add origin <url> — connect GitHub repo

git push origin <branch> — upload commits

git pull origin <branch> — fetch + merge

git fetch — download updates without merging

git remote -v — check connected remotes

Tip: pull/fetch regularly so your branch does not drift too far from the team's latest code.

Undoing Changes — Use the Right Tool Carefully

Git can undo mistakes, but choose the command carefully.

Soft reset

git reset --soft HEAD~1
Remove the last commit, but keep the changes staged. Useful when you committed too early.

Hard reset

git reset --hard HEAD~1
Remove the commit AND discard changes. Destructive — use carefully.

Restore one file

git restore <file>
Discard local edits in one file. Older tutorials may show git checkout -- <file>.

Safe team rule

For commits already pushed and used by teammates, prefer git revert instead of rewriting shared history.

Push to GitHub — Practical Setup: Step 1

Create the remote repository on GitHub before connecting your local project.

1

Open GitHub → New repository

Use your GitHub account and choose **New repository**.
This creates the empty remote repository.

2

Name and visibility

Repository name: **MyProject**
Choose Public or Private based on project needs.

3

Avoid unnecessary first conflict

If your local project already has files**, do not initialize remote with README.**
It can create an avoidable first merge conflict.

4

Copy the repository URL

Use HTTPS or SSH from GitHub.
You will use this URL as the remote origin.

Example remote URL

```
https://github.com/your-username/MyProject.git
```

Push to GitHub — Practical Setup: Steps 2–3

Open a terminal in the project folder and initialise Git if needed.

What students should do

Open Git Bash, PowerShell, Terminal or your IDE terminal.

Move into the project folder before running Git commands.

If the project is not already tracked by Git, run git init.

Commands

```
cd /path/to/your/local/repository

git init
```

Important note

git init creates the hidden .git directory that stores the local repository history. If the folder was cloned before, do not run git init again.

Key idea: work inside the correct project folder before running Git commands.

Push to GitHub — Practical Setup: Steps 4–5

Check what Git sees, stage the correct files, and create the first commit.

Commands

```
git status

# review .gitignore first

git add .

git commit -m "Initial project setup"
```

git status shows modified, untracked and staged files.

Review .gitignore before git add .

Do not stage generated folders like node_modules or build output.

Use a clear first commit message.

Commit reminder

A commit saves the staged snapshot locally. Your teammates cannot see it until you push it to GitHub.

Push to GitHub — Practical Setup: Step 6

Connect the local repository to the GitHub repository using a remote named origin.

Commands

```
git remote add origin https://github.com/your-username/MyProject.git

                                OR
git remote add origin https://<your-github-username>:<your-personal-access-token>@github.com/your-username/MyProject.git

# if origin (remote-url) already exists

git remote set-url origin https://github.com/your-username/MyProject.git
```

What it means

- origin is the conventional name for the primary remote repository.
- Use remote set-url when the remote already exists.

Security note

Do not embed personal access tokens directly inside repository URLs or screenshots. Use Git Credential Manager, SSH keys or GitHub CLI.

Push to GitHub — Practical Setup: Step 7

Push the local main branch and set the upstream tracking relationship.

First push

```
git branch -M main

git push -u origin main
```

What -M means

-M this command renames the current branch to main.

if your current branch is *master*, git branch -M main changes it to **main**.

What -u means

-u means --set-upstream.

It connects your local main branch with the remote origin/main branch.

After the first command: git push -u origin main

future pushes/ pulls can simply be: git push / git pull

Clone from GitHub — Practical Setup: Steps 1–2

Use cloning when the project already exists on GitHub and you need a local copy.

1

Copy the repository URL

Open the repository on GitHub.
Click Code and choose HTTPS or SSH.
Copy the clone URL.

2

Open your terminal

Windows: Git Bash or PowerShell.
macOS/Linux: Terminal.
IDE option: VS Code or Visual Studio.

Example HTTPS URL

```
https://github.com/your-username/repository-name.git
```

Clone from GitHub — Practical Setup: Steps 3–4

Choose where the repository folder should be created, then run git clone.

Commands

```
cd /path/to/your/folder
```

Public Repository

```
git clone https://github.com/your-username/repository-name.git
```

What git clone downloads

The project files, branches, commits and full Git history.

Common mistake

Do not create the repository folder manually and clone inside it unless you know exactly what you are doing.

Key idea: clone once, then use git pull to get future updates.

Clone from GitHub — Private Repository Setup

Private repositories require authentication before GitHub allows access.

HTTPS

Use Git Credential Manager or a secure authentication prompt. A fine-grained PAT may be used when required.

SSH

Configure an SSH key and clone using the SSH URL. This is good for frequent GitHub work.

GitHub CLI

Use gh repo clone owner/repository after signing in with GitHub CLI.

Use a Personal Access Token (PAT)

If the repository is private, GitHub requires a PAT instead of a password.

```
git clone https://your-username:your-personal-access-token@github.com/your-username/repository-name.git
```

Clone from GitHub — Verify the Local Repository

After cloning, confirm that the repository is connected correctly before starting work.

Commands

```
cd repository-name

git status

git remote -v

git switch -c feature/task-name
```

What to check

cd repository-name moves into the cloned project folder.

git status confirms branch and working tree state.

git remote -v verifies that origin points to the correct GitHub repository.

Create a feature branch before editing code.

Ready to work: pull → branch → commit → push → PR.

The Everyday Git Workflow

1	**Pull latest**	`git pull`	Start from the team's newest code
2	**Create a branch**	`git switch -c feature/x`	Work in isolation, never on main
3	**Make changes**	`edit files`	Build the feature or fix
4	**Stage & commit**	`git add . && git commit -m`	Save labelled snapshots often
5	**Push the branch**	`git push -u origin feature/x`	Share your branch on GitHub
6	**Open a pull request**	`on GitHub`	Request review & merge into main

Writing Good Commit Messages

```
# ✗ Useless messages

fix
stuff
asdfgh
final fix v2 PLEASE WORK

# ✓ Clear, conventional messages

feat: add product search endpoint
fix: prevent negative cart quantity
refactor: extract auth into service
docs: update API setup instructions
```

Rules of thumb

- Summarise WHAT & WHY, not how
- Imperative mood: 'add', 'fix', not 'added'
- ~50 chars in the subject line
- One logical change per commit
- Conventional prefixes: feat / fix / docs / refactor / test / chore
- Reference issues: 'fix: ... (#42)'

Your commit history is documentation a teammate (or future-you) reads to understand HOW the project evolved. Write it for them.

Common Git Mistakes & How to Avoid Them

Committing secrets	Passwords/API keys pushed to GitHub	Use .gitignore + env vars; never commit .env; rotate if leaked
Committing huge / generated files	node_modules, build output, datasets bloat the repo	Add them to .gitignore from day one
Working directly on main	One bad commit breaks everyone	Always branch; protect main on GitHub
Giant, vague commits	Impossible to review or revert	Small, frequent, well-named commits
Not pulling before work	Avoidable merge conflicts	git pull at the start of every session
Force-pushing shared branches	Overwrites teammates' history	Never force-push main/shared branches

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 In one sentence: what is the difference between Git and GitHub?

Q2 You committed your work locally but a teammate says they can't see it. What did you forget?

Q3 Why should you never work directly on the main branch in a team project?

Q4 You accidentally pushed a file containing an API key. What is the correct response?

PART 2

Branching Strategies & Collaborative Development

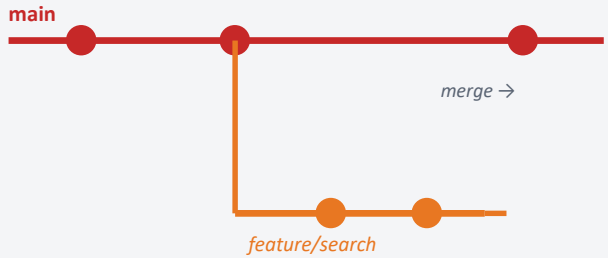
- Why branching matters; types of branches
- GitFlow · GitHub Flow · trunk-based development
- Choosing a strategy for your project
- Working effectively in a team on GitHub

Why Branching and Types of Branches

- **A branch is an isolated line of work that won't affect others until merged**
  - Enables parallel features, safe experiments and stable releases
  - main never breaks — work happens on branches and merges back when ready

`main / master`	Always-deployable, stable production code
`feature/*`	New functionality (feature/product-search)
`bugfix/* (fix/*)`	Non-urgent fixes during development
`release/*`	Stabilising a version before shipping
`hotfix/*`	Urgent production fixes, branched from main

Branch & merge



Branch off main → commit in isolation → merge back when reviewed & tested.

Branching Strategies

GitFlow

- **GitFlow** is a more structured branching strategy.
- It mainly uses two long-term branches:
  - **main** for production-ready code
  - **develop** for ongoing development
- New features are created using **feature**/* branches from develop.
- Before release, teams use a **release**/* branch for final testing and fixes.
- Urgent production problems are handled using **hotfix**/* branches from main.
- **Strengths: clear structure; supports scheduled, versioned releases**
- **Trade-offs: complex; many long-lived branches; slower for web apps**
- Best for: versioned/desktop software, multiple releases in support



feature → develop → release → main

Branching Strategies

GitHub Flow

- **One main branch — always deployable**
- Branch → commit → open PR → review → merge → deploy
- Short-lived feature branches (hours/days)
- Every merge to main can ship immediately
- Simple, fast, ideal for web apps & small teams
- Relies on strong CI + reviews to keep main green

Trunk-Based Development

- **Everyone commits to one trunk very frequently**
- Tiny, short-lived branches merged daily
- Incomplete work hidden behind feature flags
- Demands excellent automated testing
- Enables continuous delivery at scale (Google, Meta)
- Hard without strong test & CI discipline

Choosing a Branching Strategy (LO4)

Aspect	GitFlow	GitHub Flow	Trunk-Based
Complexity	High	Low	Low–Medium
Long-lived branches	Many (main, develop, …)	One (main)	One (trunk)
Release style	Scheduled, versioned	Continuous	Continuous
Needs strong CI/tests	Helpful	Important	Essential
Best for	Versioned / desktop software	Web apps, small teams, students	Large teams at scale

For your SE3090 assignment: GitHub Flow is the sensible default — simple, review-driven, and a great fit for a small team building a web app. Justify your choice (LO4).

Working Effectively in a Team on GitHub

Protect main

- Branch protection: no direct pushes
- Require PR + passing CI + ≥1 review
- Keeps the shared branch always green

One branch per task

- Small, focused, short-lived branches
- Clear naming: feature/ fix/ chore/
- Delete branches after merge

Sync often

- Pull/rebase main into your branch daily
- Avoid long-lived divergence
- Smaller, safer merges

Communicate & agree

- Define the workflow on day one
- Use issues + a board to coordinate
- Review promptly so PRs don't pile up

PART 3

Pull Requests, Code Review & Issue Tracking

- What a pull request is and the PR workflow
- Reviewing code: a practical checklist
- Handling review comments & merge conflicts
- GitHub Issues: labels, milestones, assignees & linking

Pull Requests — Propose, Review, Merge

- **A pull request (PR) proposes merging one branch into another**
  - A place to review, discuss and improve changes BEFORE they hit main
  - Shows the diff, runs CI checks, and records the conversation
- **The quality gate of collaborative development**

```
feature/product-search  →  main
✓ All checks passed
✓ 1 approval
+182 −7 · 3 files
Linked: closes #42
```

1. **Push a feature branch**
   branch is on GitHub
2. **Open a PR → main**
   describe what & why; link issue
3. **CI runs automatically**
   build, tests, lint, security
4. **Teammates review**
   comments, suggestions, approve/request changes
5. **Address feedback**
   push fixes; discussion resolves
6. **Merge & delete branch**
   main updated; branch cleaned up

Reviewing Code Before Merging

- **Code review = means another developer checks your changes before they are merged.**
  - Goals: catch bugs, share knowledge, keep the codebase consistent
- **Review the CHANGE, not the person — be kind, specific, constructive**
- **GitHub review actions:**
  - ***Comment*** — give feedback without blocking the PR.
  - ***Approve*** — the changes are acceptable to merge.
  - ***Request changes*** — important issues must be fixed first.
- **Small PRs get better reviews — aim for < ~400 lines changed**

Three review outcomes

💬 Comment

✓ Approve

✗ Request changes

```
// a GOOD review comment
Nit: extract this into a helper —
it's repeated in 3 places. Also,
what happens if `items` is empty?
Suggest a guard clause here.
```

A Practical Code Review Checklist

Correctness

- ✓ Does it do what the PR says?
- ✓ Edge cases & error paths handled?
- ✓ Tests added and passing?

Design & readability

- ✓ Clear names; single responsibility?
- ✓ Is there unnecessary duplication?
- ✓ Is the code easy to understand?

Security

- ✓ No secrets/keys committed?
- ✓ Inputs validated; no injection?
- ✓ Authorization checked server-side?

Performance & scope

- ✓ No obvious N+1 / heavy loops?
- ✓ PR stays small & focused?
- ✓ No unrelated changes sneaked in?

Merge Conflicts — Why Git Stops the Merge

A merge conflict is Git asking the developer to decide between overlapping changes.

1. **Two branches change code**
   main and feature-branch both edit index.html.

2. **Git tries to merge**
   Non-overlapping changes are merged automatically.

3. **Overlap found**
   Git stops and marks the file for manual resolution.

```
git checkout main
git merge feature-branch
# Git stops if the same line was changed differently
```

Key idea: conflicts are normal in team projects — resolve carefully, then continue the merge.

Merge Conflict Example

Alice and Bob edit the same line in index.html in different branches.

Alice — feature branch

index.html

<h1>Welcome to Our Website!</h1>

Bob — main branch

index.html

<h1>Welcome to My Website!</h1>

Steps leading to conflict

1. Bob commits his version to main.
2. Alice switches to main.
3. Alice runs git merge feature-branch.
4. Git detects the same line changed differently.

```
git checkout main
git merge feature-branch
```

Merge Conflict Example — Conflict Markers

Git writes conflict markers into the file so the developer can see both versions.

```
<<<<<<< HEAD
<h1>Welcome to My Website!</h1>
=======
<h1>Welcome to Our Website!</h1>
>>>>>>> feature-branch
```

<<<<<<< HEAD
Current branch version — main / Bob.

=======
Separator between the two versions.

>>>>>>> feature-branch
Incoming branch version — Alice.

```
git status
```

Use git status to identify all conflicted files before fixing them.

Merge Conflict Example — Resolution Choices

The correct choice depends on the required behaviour, not on who changed it first.

1. Accept Current

Keep Bob's main version.

<h1>Welcome to My Website!</h1>

2. Accept Incoming

Keep Alice's feature version.

<h1>Welcome to Our Website!</h1>

3. Merge Manually

Rewrite into the final correct version.

<h1>Welcome to Our Website</h1>

```
# after choosing the final line, remove all markers
# then save the file
```

Important: never leave <<<<<<<, ======= or >>>>>>> markers inside the final code.

Merge Conflict Example — Finalise the Fix

After editing the file, tell Git that the conflict is resolved and commit the merge.

```
git add index.html
git commit -m "Resolved merge conflict in
index.html"
git push origin main
```

Final check before pushing

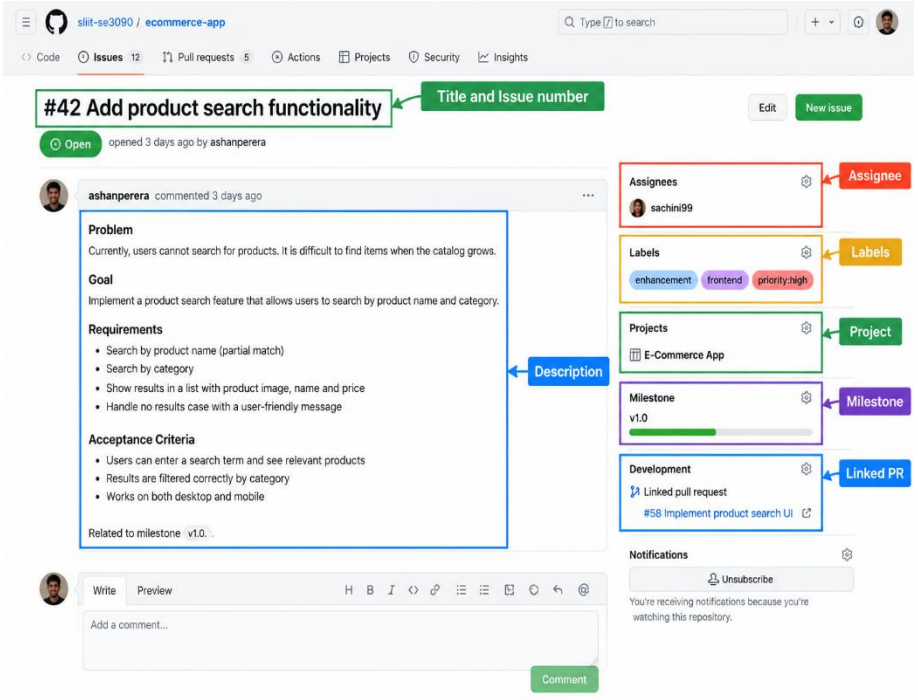
- Run the application or tests.
- Review the final file content.
- Confirm no conflict markers remain.
- Then push the resolved merge.

To reduce conflicts

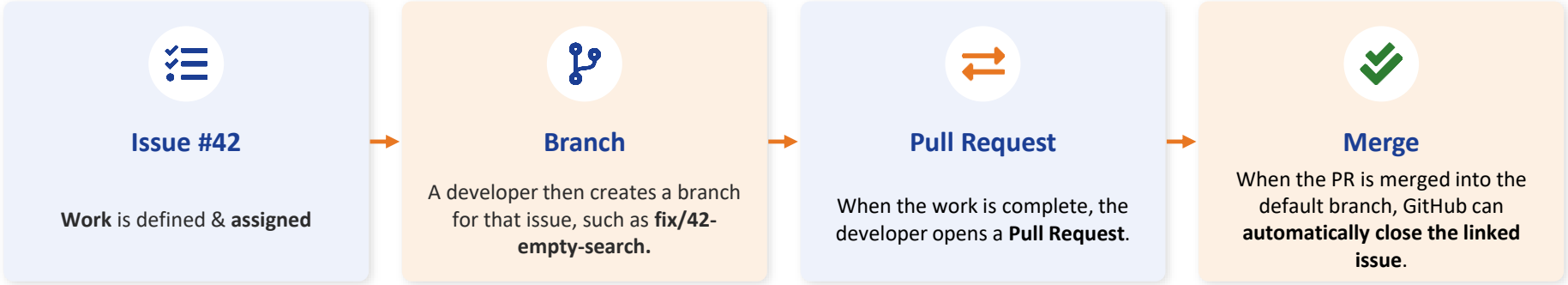
1. Pull latest changes before work.
2. Keep branches and PRs small.
3. Communicate when editing same files.
4. Merge frequently when suitable.

Tracking Work with GitHub Issues

- **An Issue = a unit of work: a bug, feature, task or question**
  - Issues act like the team's shared to-do list inside the repository.
  - **Each issue has a title, description, comments & a number (#42)**
- **Use labels to classify work, for example:**
  - Labels — bug, enhancement, priority:high, good-first-issue
  - Assignees — to show who is responsible for the task.
  - Milestones — group issues into a release/sprint
- GitHub Projects can show issues on a Kanban board: To Do → In Progress → Done.
- Issues can also be linked to PR.



Linking Issues & Pull Requests



Magic keywords in a PR or commit

`closes #42` · `fixes #42` · `resolves #42` → GitHub auto-closes the issue when the PR merges. If you only write #42 without a keyword, GitHub links the PR and issue but does not automatically close it.

Result: Full Traceability — what problem existed | which code fixed it | who reviewed it |when it was merged.

Professional Collaborative Development

Protect main

Require PRs + passing CI + approval before merge. No direct pushes to main.

Keep the PR Small and frequent

Easier to review, faster to merge, fewer conflicts. One concern per PR.

Communicate openly

Use issues, PR threads & standups. Make work and blockers visible to the team.

Keep history clean

Meaningful commits & PR titles; squash noisy WIP commits when merging.

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we compare answers

D1 A teammate opens a 2,000-line PR titled "updates". List three things wrong and how you'd fix the process.

D2 You and a teammate edited the same function and now have a conflict. Walk through exactly how you resolve it safely.

D3 What's the difference between an Issue and a Pull Request — and how should they be linked?

D4 Your team keeps pushing straight to main and breaking it. What ONE GitHub setting fixes this, and what does it enforce?

PART 4

Continuous Integration & Continuous Delivery (CI/CD)

- CI vs Continuous Delivery vs Continuous Deployment
- Pipeline stages: build → test → quality → security → deploy
- GitHub Actions overview
- Why CI/CD transforms team productivity & quality

CI, Continuous Delivery & Continuous Deployment



Continuous Integration

means developers merge changes frequently, and every push automatically runs the build and tests.
The goal of CI is to detect integration problems early, before they become bigger issues.



Continuous Delivery

goes one step further:

- the application is automatically built, tested, and prepared for release
- but a **human still decides when to deploy**



Continuous Deployment

removes that final manual step.

- If all checks pass, the new version is **automatically deployed to production**.
- So the main difference between Delivery and Deployment is the manual approval step.

CI is a prerequisite for both deliveries — you can't safely automate releases without automated tests.

Why CI/CD improves team productivity and software quality.

Catch bugs early

Defects are found minutes after a push, when they're cheap to fix — not weeks later in production.

Ship faster & safer

Automated pipelines release in minutes with consistent, repeatable steps — no error-prone manual deploys.

Consistent quality

Every change passes the SAME build, tests, linting and security checks. No 'works on my machine'.

Confidence to change

A safety net of automated checks lets the team refactor and add features without fear of silent breakage.

Anatomy of a CI/CD Pipeline

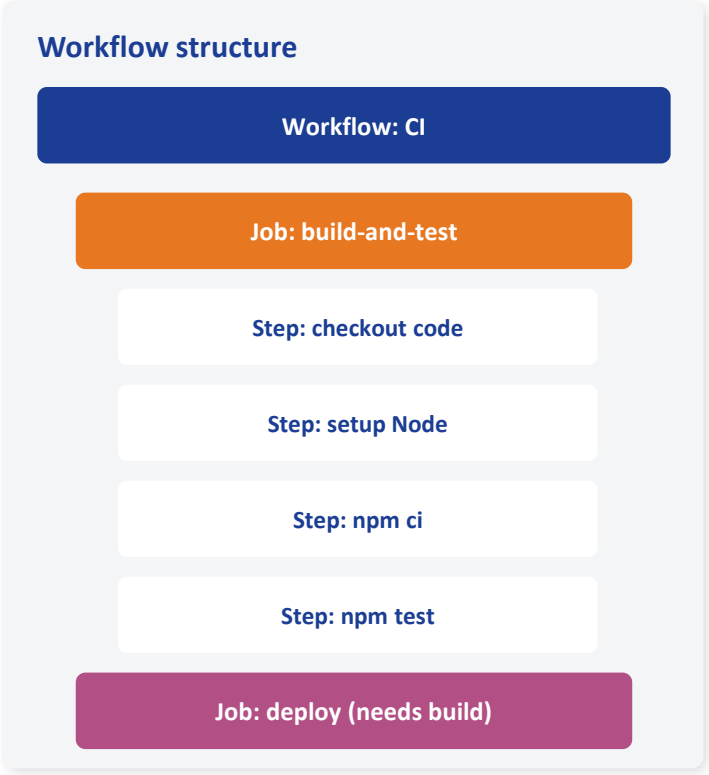


Key principles

- **Fail fast — if one stage fails, stop the pipeline immediately**
- Each stage is automated and identical for every change; later stages cost more, so cheap checks run first
- A green pipeline = the change is safe to merge / release

GitHub Actions — CI/CD Built Into GitHub

- **GitHub Actions workflows are stored inside the repository under:**
  - Workflows are YAML files in .github/workflows/
- **Triggered by events: push, pull_request, schedule, manual…**
- **Core vocabulary:**
  - **Workflow** — the whole automated process
  - **Job** — a group of steps that run together
  - **Step** — a single command or reusable action
  - **Runner** – the machine where the job runs
- GitHub also provides a **Marketplace** with reusable actions, such as checking out code or setting up Node.js.



Example: A CI Workflow (GitHub Actions)

.github/workflows/ci.yml

```yaml
name: CI
on:                          # when to run
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  build-and-test:
    runs-on: ubuntu-latest   # a fresh runner
    steps:
      - uses: actions/checkout@v4        # gets the repository code
      - uses: actions/setup-node@v4      # installs Node.js
        with: { node-version: 20 }
      - run: npm ci                      # installs dependencies
      - run: npm run lint                # check the code quality
      - run: npm test                    # runs tests
      - run: npm audit --audit-level=high # security
```

Reading it

- **on**: runs on every push & PR to main
- Under jobs, we define a job called **build-and-test**.
- **runs-on**: ubuntu-latest means GitHub creates a fresh Ubuntu runner for the job.
- checkout → get the repo
- setup-node → install the toolchain
- npm ci → reproducible install
- lint → test → audit = quality + security gates
- If any step fails, the workflow fails and the PR shows a **red check** ❌.

Automated Build & Testing in the Pipeline

Automated build

- means the application is **compiled** or **bundled** from a clean checkout every time.
- This helps catch problems such as:
  - missing filesincorrect
  - Dependencies
  - machine-specific configuration“
  - works on my machine” issues
- Reproducible: same steps, same result
- If the build fails, the pipeline should stop immediately.- *fix before anything else*

Automated testing

- checks whether the software behaves correctly.
- ***Unit tests*** — one function/component in isolation (fast, many)
- **Integration tests** — modules working together (e.g. API + DB)
- These tests should run on every push or Pull Request so regressions are detected quickly.
- **Code coverage** shows how much of the code is exercised by tests.

Handling Build & Test Failures

Pull request checks

✓ build

✓ lint

✗ test (2 failed)

• deploy — skipped

Merge blocked ✗

- **First**, open the CI logs and identify the exact failing step and error message.
- Try to reproduce the problem locally.
- Fix the issue, commit the correction, and push again.
- GitHub Actions will automatically run the pipeline again.
- Keeping main green is a shared team responsibility.
- A broken main branch should be treated as a ***high-priority issue*** because it can block the whole team.

Red pipeline = STOP. Don't merge, don't build on top of it

The early-defect-detection payoff

- A bug caught in CI costs minutes; the same bug in production costs hours of downtime, hotfixes & lost trust
- Fast feedback means you fix it while the change is still fresh in your mind
- Automated gates stop one person's broken code from blocking the whole team

Checkpoint 03 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 Continuous Delivery vs Continuous Deployment — what is the single difference?

Q2 In GitHub Actions, what is the difference between a workflow, a job and a step?

Q3 Your PR's test stage fails. What should you do, and what must you NOT do?

Q4 Why do cheap checks (build, lint) run before expensive ones (integration tests, deploy)?

PART 5

Code Quality & Clean Coding Practices

- What 'code quality' really means
- Clean code principles: naming, duplication, function design
- Static analysis, linting & formatting
- Technical debt & refactoring

What Is Code Quality?

- **Quality code works — and keeps working as it's changed by others**
  - It's judged by how easy it is to read, change & trust, not just 'it runs'

Readability

Another developer (or you in 6 months) can understand it quickly.

Maintainability

Easy and safe to change, extend and fix without breaking things.

Reliability

Behaves correctly, handles errors, and is covered by tests.

Scalability

Holds up as data, users and the team grow.

Clean Code Principles

```
// ✗ unclear, does too much

function d(x, y) {
  return x.p * y - x.p * y * 0.1;
}

// ✓ intention-revealing & focused

function applyDiscount(
    product, qty) {
  const subtotal = product.price * qty;
  return subtotal - subtotal * DISCOUNT;
}
```

- **Meaningful names —** say what & why, not x, tmp, data2
- **Small functions, one responsibility (SRP)**
- **DRY — don't repeat yourself; extract shared logic**
- **Avoid magic numbers — use name constants (DISCOUNT = 0.1)**
- **Keep nesting shallow; prefer guard clauses**
- **Comment WHY, not WHAT the code already says**
- **Consistent style across the whole codebase**

The test: could a teammate read this function once and know exactly what it does and why? Clean code optimises for the READER, because code is read far more than it is written.

Static Analysis, Linting & Formatting

Static analysis

Tools inspect code **WITHOUT running it**

It can find: possible bugs |code smells |risky patterns

Linting

is a type of static analysis that checks coding rules and common mistakes

Examples include: unused variables | == instead of === |missing *await* |unreachable code

Formatting

automatically keeps the code style consistent.

spacing | indentation | quotes |line length

In CI

these checks can run automatically on every Pull Request.

This means code quality is **enforced consistently by tools**, not left to personal preference.

Why automate style? It ends pointless review debates about spaces vs tabs, frees humans to review LOGIC, and keeps diffs focused on real changes.

Tools: ESLint, Prettier, SonarQube & CodeQL

Tool	Does	Catches / provides
ESLint	Lints JS/TS code	Bugs, bad patterns, style rule violations
Prettier	Auto-formats code	Consistent spacing, quotes, line width — no debates
SonarQube	Deep code-quality platform	Bugs, code smells, duplication, coverage, security hotspots, tech-debt rating
CodeQL	Semantic security analysis (GitHub)	Vulnerabilities like injection & data-flow flaws across the codebase

Everyday pair: ESLint (correctness/style) + Prettier (formatting) in every JS/TS project & CI.

Heavier guns: SonarQube (quality dashboards) & CodeQL (security scanning) plug into the pipeline for deeper analysis.

Technical Debt & Refactoring

Technical debt

- Technical debt means taking shortcuts now that create extra cost later.
- It is similar to financial debt:
  - a small amount may be manageable
  - but if ignored, the “interest” keeps growing
- *Common causes include:*
  - rushed code
  - copy-paste duplication
  - skipped tests
  - outdated dependencies
  - TODO items that are never fixed
- A common symptom is that **every new change becomes slower and riskier.**

Refactoring

- Refactoring means improving the **internal structure** of the code **without changing its behaviour**.
- Examples:
  - rename unclear variables
  - extract methods
  - remove duplication
  - simplify complex logic
- Tests are important because they give confidence that refactoring has not broken functionality.
- A good habit is to improve the code continuously, instead of waiting for one big cleanup.

PART 6

Security Risks & Secure Coding Practices

- Common security risks in software & repositories
- Security Risks Specific to Repositories

Common Security Risks in Software Projects

Hardcoded credentials & exposed API keys	Passwords/keys in source or commits — scraped by bots within minutes of being pushed.
Weak auth & broken access control	Guessable passwords, no MFA; users reaching data/actions they shouldn't (OWASP #1).
SQL injection & XSS	Untrusted input run as a query or script — data theft or hijacked sessions.
Insecure dependencies	Known-vulnerable packages pulled in transitively — most of your code is other people's.
Poor error handling & config	Stack traces & secrets leaked in errors; debug mode or default creds in production.

Security Risks Specific to Repositories

```
the classic leak

// ✕ NEVER hardcode secrets

const db = connect(
  'postgres://admin:P@ss123@prod-db/app'
);
const STRIPE_KEY = 'sk_live_4eC39...';

// ✓ read from the environment

const db = connect(process.env.DB_URL);
const STRIPE_KEY = process.env.STRIPE_KEY;
```

- **The biggest risk is secrets committed to code or git history**
- **.env files pushed instead of git-ignored**
- **Public repos exposing internal code & config**
- **Over-permissive tokens / deploy keys**
- **Unvetted dependency updates merged blindly**
- **Bots scan public GitHub continuously — leaks are found fast**

If a secret IS leaked

- **ROTATE (**replace the old secret with a new one.) **it immediately (assume it's already compromised) — deleting the commit is NOT enough; git history keeps it**
- Add the file to .gitignore; use GitHub secret scanning & push protection to prevent the next one

PART 7

Secure Coding & Dependency Management

- The secure-coding mindset: never trust input
- Validation, sanitization & safe error handling
- Secrets via environment variables
- Dependency scanning & automated security in CI/CD

Secure Coding: Never Trust Input

- **Treat ALL external input as hostile until proven safe**
  - Users, APIs, files, query params, headers — all untrusted
- **Validate — reject anything that doesn't fit the expected shape**
- **Sanitize / encode — neutralise dangerous characters for the context**
- **Use safe APIs: parameterized queries, framework escaping**
- **Defence in depth: validate on client AND server; the server is the real gate**
- **Fail securely — deny by default, reveal nothing useful in errors**

```
// ✗ SQL injection
query("SELECT * FROM users WHERE" +
      " email='" + input + "'");

// ✓ parameterized — input is DATA
query("SELECT * FROM users" +
      " WHERE email = $1", [input]);

// ✓ validate shape first
if (!isEmail(input)) return 400;
```

Same data, two worlds: concatenation lets input become CODE; parameters keep it as DATA the DB can't misread.

Secrets Management & Secure Configuration

Never in code

No passwords, keys or tokens in source or git history. Ever.

Use env variables

Read secrets from process.env / config; keep .env git-ignored.

CI/CD secrets store

GitHub Actions Secrets inject values at run time — encrypted, never printed.

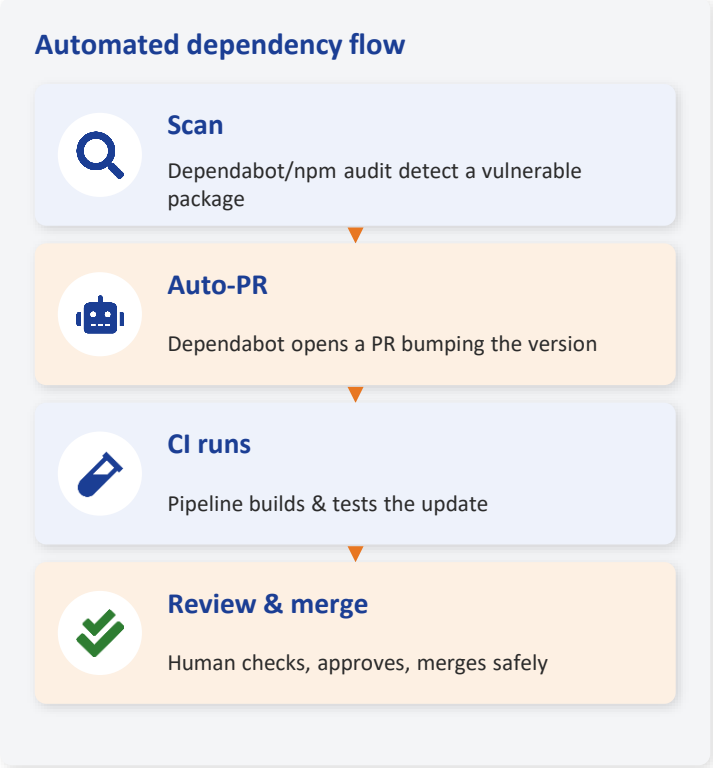
Per-environment config

Different secrets for dev/test/prod; production secrets in a vault.

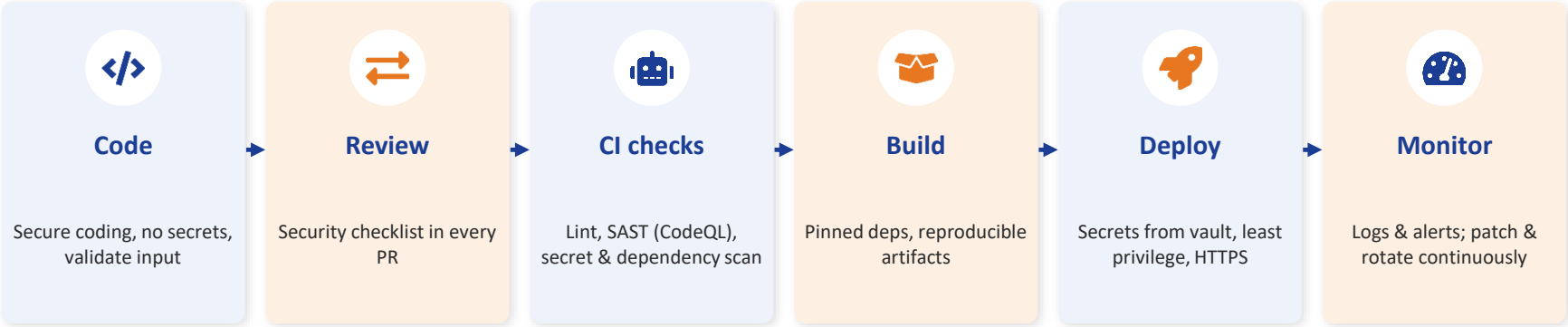
```
.gitignore          # commit this
.env                # NEVER commit this
.env.local
# In GitHub Actions: ${{ secrets.DB_URL }}
```

Dependency Management & Vulnerability Scanning

- **Most of a modern app is third-party code — their bugs are YOUR risk**
  - Vulnerabilities are found in popular packages constantly
- **Keep dependencies updated — but review changes, don't merge blindly**
- **Use automated tools such as :**
  - npm audit — lists known vulnerabilities & severities
  - GitHub Dependabot — auto-PRs to bump vulnerable deps
  - CodeQL / code scanning — finds vulnerable code patterns
- Use a **lockfile** such as ***package-lock.json*** to keep dependency versions consistent and reproducible**.**



Security Across the Development Lifecycle



"Shift left" — build security in from the start

- Security isn't a final step — it's woven through every stage of the lifecycle
- Automated gates (secret, dependency & code scanning) run in CI on every PR
- Cheaper and safer to prevent vulnerabilities than to patch breaches

Case Study: LankaMart Team on GitHub + CI/CD



How a 4-person team ships safely (practices → why)

Practice	What the team does	Why
Branching	GitHub Flow: short feature branches off main	Simple for a small team; continuous delivery-friendly
Planning	GitHub Issues + project board; labels & assignees	Visible, owned work; no lost tasks
Quality gate	PRs require CI green + 1 approval to merge	Nothing untested or unreviewed reaches main
CI pipeline	Actions: build → lint → test → npm audit	Catches bugs, style & vulnerable deps automatically
Security	Env vars + GitHub Secrets; Dependabot on	No secrets in code; dependencies stay patched
Traceability	Every PR links an issue ("Closes #__")	Full history of what changed, why & who reviewed

Group Activity: Design Your Team Workflow (20 min)

Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: 4-person LankaMart team, 6 weeks

You're starting the full-stack project (React + ASP.NET Core + PostgreSQL) on GitHub.

- **Task 1 — Pick a branching strategy & justify it for a small team**
- **Task 2 — Define your PR rules (reviews, branch protection, merge policy)**
- **Task 3 — Sketch a CI pipeline: list the stages on every PR**
- **Task 4 — Name three security measures you'll enforce from day one**

Marking lens

- Strategy fits a small team (GitHub Flow), with reasons?
- main protected: PR + CI + approval?
- Pipeline has build, test, quality & security stages?
- Security: no secrets in code, env vars, dependency scan?
- Choices justified, not just named (LO4)?

Activity: Review This Pull Request

Spot the issues — 5 min in pairs. How many can you find?

login.js (+10 −0)

```
// PR: "add login"  (changes login.js)
const API_KEY = 'sk_live_91x_REALKEY';

function login(u, p) {
  const q = "SELECT * FROM users WHERE" +
          " name='" + u + "' AND pass='" + p + "'";
  const r = db.run(q);
  if (r) { return r; }   // returns full user row
  // no error handling, no validation
}
```

Hunt for…

- A hardcoded secret
- An injection vulnerability
- Plaintext password handling
- Over-exposed data in the response
- Missing validation & error handling
- Poor naming & no tests

Write your review comments as if on GitHub — be specific, kind and actionable.

Quick Quiz — 6 Questions

1	What does a pull request let a team do before code reaches main?	A) deploy B) review & run checks C) delete history
2	CI mainly automates which activity on every push?	A) design B) build & test C) marketing
3	You leaked an API key in a commit. First action?	A) delete the commit B) rotate the key C) ignore it
4	Best defence against SQL injection?	A) hide errors B) parameterized queries C) longer passwords
5	Which tool auto-opens PRs to fix vulnerable dependencies?	A) Prettier B) Dependabot C) ESLint
6	For a small 4-person team, the simplest fitting branching model is…	A) GitFlow B) GitHub Flow C) no branches

Lecture Summary — Key Takeaways

Git is your team's backbone

Branches, commits & remotes enable safe parallel work and a full history.

CI/CD automates the checks

Build, test, quality & security run on every change — catch bugs early.

PRs + reviews gate quality

Nothing reaches main untested or unreviewed; reviews share knowledge.

Quality is engineered

Clean code, linting, formatting & paying down technical debt.

Issues make work visible

Track, label, assign & link to PRs for full traceability.

Security is everyone's job

No secrets in code, validate input, scan dependencies — shift left.